

6DoF Motion Deblurring — Implementation Plan

6DoF Motion Deblurring — Implementation Plan

Project: Subsea Rover 3D Scanner
Stack: ROS 2 Jazzy · Raspberry Pi 5 · BNO085 IMU · simpleRTK3B · Dual RPi Global Shutter Cameras
Date: May 2026

Current State

The pipeline already implements 2.5DoF deblurring:

Component	Status
Gyro integration (pitch + yaw → pixel blur vector)	Done
Translational blur from odometry velocity	Done (enable with <code>deblur_use_translation:=true</code>)
IMU-to-camera yaw correction	Done (tune <code>deblur_imu_to_cam_yaw_deg</code>)
Richardson-Lucy deconvolution	Done
Wiener deconvolution (FFT, faster)	Done
Roll axis contribution	Missing
Full rotation matrix integration	Missing
Spatially-varying PSF (per-pixel homography)	Missing
Camera intrinsic calibration (K matrix)	Not calibrated
Camera-IMU extrinsic calibration (R, t)	Not calibrated
Camera-IMU time offset calibration	Not calibrated
RTS smoother for post-processing	Not implemented

The gap between current state and true 6DoF is primarily **spatially-varying PSF** and **extrinsic calibration**. Everything else is incremental improvement.

Phase 1 — Camera Intrinsic Calibration

What it gives you: Accurate focal length and principal point for the homography-based PSF in Phase 3. Currently the pipeline uses a single `deblur_fov_deg` value and assumes the principal point is at the image centre. Real lenses deviate from this.

Required: A printed checkerboard pattern (7×9 inner corners, 25 mm squares), mounted flat on a rigid surface.

Procedure:

1. Install the ROS calibration tool:

```
sudo apt install ros-jazzy-camera-calibration
```

2. Start each camera individually and run calibration — move the checkerboard slowly in front of the camera, covering all corners of the frame:

```
ros2 run camera_calibration cameracalibrator \  
  --size 7x9 --square 0.025 \  
  image:=/cam0/camera/image_raw \  
  camera:=/cam0/camera
```

Collect at least 40 images with the board at varying distances (0.3–1.5 m), tilts, and positions. The tool shows a progress bar for X, Y, size, and skew coverage. Click **Calibrate** when all bars are green, then **Save**.

3. Repeat for cam1.
4. The output `ost.yaml` contains:

```
camera_matrix:  
  data: [fx, 0, cx, 0, fy, cy, 0, 0, 1]  
distortion_coefficients:  
  data: [k1, k2, p1, p2, k3]
```

Note f_x , f_y (focal lengths in pixels) and c_x , c_y (principal point).

5. Update `deblur_fov_deg` to match the calibrated focal length:

```
import math  
fx = 660.0 # from calibration  
image_width = 4056  
fov_deg = 2 * math.degrees(math.atan(image_width / (2 * fx)))
```

6. Store the full K matrix for use in Phase 3. Add `cam0_calib.yaml` and `cam1_calib.yaml` launch parameters pointing to the calibration files.

Time estimate: 1–2 hours per camera including setup.

Phase 2 — Camera-IMU Time Offset and Extrinsic Calibration

This is the most critical calibration step. Even a 5 ms timestamp error at 1 rad/s rotation causes ~0.5 px blur direction error. An unknown camera-IMU rotation causes a systematic blur direction error for every capture.

2a — Time offset calibration

What it gives you: The fixed latency `t_offset` between the camera frame timestamp and actual mid-exposure. Caused by driver delays, USB/CSI buffering, and OS scheduling.

Procedure:

1. Mount the camera looking at a point LED or laser spot that you can blink at known times using a GPIO pin.
2. Record a bag while blinking the LED at 5 Hz. The LED blink appears as a sudden brightness step in the image, and the GPIO event has a precise system timestamp.
3. Cross-correlate the brightness time series extracted from the bag with the GPIO event timestamps. The lag is `t_offset`.
4. Alternatively: use the **continuous motion method** — shake the camera at ~2 Hz while recording IMU and camera. Apply the deblur with different time offsets (–20 ms to +20 ms in 1 ms steps) and find the offset that minimises residual blur in the output image sharpness metric (variance of Laplacian).
5. Set the result as a fixed offset in `_deblur_exposure_window_ns`:

```
ros2 launch ... deblur_timestamp_offset_ms:=5.0
```

Time estimate: 2–3 hours.

2b — Camera-IMU extrinsic calibration (rotation + translation)

What it gives you: The rotation matrix `R_cam_imu` that transforms angular velocity from the IMU frame into the camera frame. Currently this is approximated by `deblur_imu_to_cam_yaw_deg` (yaw-only). The full 3×3 rotation handles pitch and roll mounting offsets too.

Tool: Kalibr (runs on desktop, not the Pi)

1. Install on your development machine:

```
docker pull stereolabs/kalibr
```

2. Record a calibration bag on the Pi — move the rover through full 6DoF motion (all axes) in front of the checkerboard pattern for 2–3 minutes at moderate speed:

```
ros2 bag record /cam0/camera/image_raw /cam1/camera/
            image_raw /imu/data \
-o kalibr_calib
```

IMU must be at 100 Hz. Camera at a reduced rate (10–20 Hz) for this bag only — Kalibr needs lower frame rates for its optimiser.

3. Run Kalibr:

```
docker run -it --rm -v $(pwd):/data stereolabs/kalibr \
kalibr_calibrate_imu_camera \
```

```
--bag /data/kalibr_calib \
--cam /data/cam_chain.yaml \
--imu /data/imu.yaml \
--target /data/checkerboard.yaml
```

4. Output: `results-imucam.txt` containing `T_cam_imu` — a 4×4 homogeneous transform. Extract the 3×3 rotation block `R_cam_imu`.
5. Add `R_cam_imu` as a 9-float parameter (`deblur_cam_imu_rotation`) and replace the current yaw-only rotation in `_estimate_blur_kernel` with the full matrix multiply:

```
R = np.array(R_cam_imu).reshape(3, 3)
omega_imu = np.array([dtheta_x, dtheta_y, dtheta_z])
omega_cam = R @ omega_imu
dx = omega_cam[0] * focal_px * strength # now correct for
    all mounting angles
dy = omega_cam[1] * focal_px * strength
```

Time estimate: 3–4 hours including Kalibr setup.

Phase 3 — Full Rotation Matrix Integration

What it gives you: Correct blur calculation for large rotations ($> \sim 0.1$ rad), and the per-sample rotation matrices needed for the spatially-varying PSF in Phase 4.

Currently the pipeline accumulates scalar angles via trapezoidal integration:

```
dtheta_x += 0.5 * (w_a[0] + w_b[0]) * dt_s
```

This is a small-angle approximation. For 8 ms exposure at 1 rad/s it introduces $< 0.1\%$ error, but at higher speeds or longer exposures it matters.

Implementation in `capture_service.py` — `_integrate_gyro_interval`:

Replace the scalar accumulation with rotation matrix integration using the Rodrigues formula:

```
import numpy as np

def _rodrigues(rvec: np.ndarray) -> np.ndarray:
    """Rodrigues rotation: angle-axis vector → 3×3 rotation
        matrix."""
    angle = float(np.linalg.norm(rvec))
    if angle < 1e-12:
        return np.eye(3)
    axis = rvec / angle
    K = np.array([[0, -axis[2], axis[1]],
                  [axis[2], 0, -axis[0]],
                  [-axis[1], axis[0], 0]])
```

```

    return np.eye(3) + math.sin(angle) * K + (1 -
        math.cos(angle)) * (K @ K)

# In _integrate_gyro_interval, replace scalar loop with:
R_total = np.eye(3)
for i in range(1, len(dedup)):
    t_a, w_a = dedup[i - 1]
    t_b, w_b = dedup[i]
    dt_s = float(t_b - t_a) / 1e9
    if dt_s <= 0.0:
        continue
    omega_avg = np.array([(w_a[0]+w_b[0])*0.5,
                          (w_a[1]+w_b[1])*0.5,
                          (w_a[2]+w_b[2])*0.5])
    R_step = _rodrigues(omega_avg * dt_s)
    R_total = R_step @ R_total

# Return the rotation matrix alongside existing diagnostics
out["R_total"] = R_total.tolist()
# For backward compatibility, also return equivalent small-angle
# values:
out["delta_theta_rad"] = [R_total[2,1], -R_total[2,0], R_total[1,
    0]]

```

Store the full `R_total` per gyro integration call. It is needed directly in Phase 4.

Time estimate: 2–3 hours including testing.

Phase 4 — Spatially-Varying PSF via Tiled Homography

What it gives you: The most significant quality improvement. Currently the same blur vector is applied to every pixel in the image. In reality, rotation creates different blur at each pixel — pixels far from the rotation centre are blurred more and in a different direction than pixels near the centre.

Model: For a camera rotating by `R_total` during exposure, a pixel at (u, v) moves according to the homography:

$$H = K \cdot R_{\text{total}} \cdot K^{-1}$$

where K is the camera intrinsic matrix from Phase 1. The pixel displacement at (u, v) is:

$$\begin{aligned}
 p' &= H \cdot [u, v, 1]^T \\
 \delta u &= p'[0]/p'[2] - u \\
 \delta v &= p'[1]/p'[2] - v
 \end{aligned}$$

This displacement varies across the image — it is zero at the principal point and largest at the corners.

Tiled implementation (practical approximation, avoids per-pixel PSF):

Divide the image into a grid (e.g. 4 columns $\times$ 3 rows = 12 tiles). For each tile, compute the blur vector at the tile centre pixel using the homography, build a PSF for that tile, and deconvolve independently.

```
def _deblur_tiled(img_bgr, K, R_total, method, snr, iterations,
                 max_kernel,
                 min_kernel, strength, grid=(4, 3)):
    h, w = img_bgr.shape[:2]
    K_inv = np.linalg.inv(K)
    H = K @ R_total @ K_inv
    rows, cols = grid
    tile_h = h // rows
    tile_w = w // cols
    result = np.zeros_like(img_bgr)

    for r in range(rows):
        for c in range(cols):
            y0, y1 = r * tile_h, (r + 1) * tile_h if r < rows -
            1 else h
            x0, x1 = c * tile_w, (c + 1) * tile_w if c < cols -
            1 else w

            # Blur vector at tile centre
            uc, vc = (x0 + x1) / 2.0, (y0 + y1) / 2.0
            p = H @ np.array([uc, vc, 1.0])
            du = p[0] / p[2] - uc
            dv = p[1] / p[2] - vc
            length_px = math.sqrt(du*du + dv*dv) * strength
            angle_deg = math.degrees(math.atan2(dv, du))

            tile = img_bgr[y0:y1, x0:x1]
            if length_px < min_kernel:
                result[y0:y1, x0:x1] = tile
                continue

            psf = _line_psf(length_px, angle_deg, max_kernel)
            if method == "wiener":
                result[y0:y1, x0:x1] =
                _wiener_deconvolve_bgr(tile, psf, snr)
            else:
                result[y0:y1, x0:x1]
                = _richardson_lucy_bgr(tile, psf, iterations)

    return result
```

Add a `deblur_tiled` boolean parameter (default `false`) and a `deblur_tile_grid` string parameter (default `"4x3"`). When `deblur_tiled=True` and `K` is provided, use `_deblur_tiled` instead of the global PSF path.

Tile boundary artefacts: Hard tile boundaries cause visible seams. Fix with 20% tile overlap and a cosine blend:

```
# Blend overlapping regions with a cosine window weight
# Each pixel's value = weighted average of all tiles that cover
  it
```

This is an additional 1–2 hours on top of the basic tiled implementation.

Time estimate: 4–6 hours total (basic tiling 2–3 h, overlap blending 1–2 h, integration 1 h).

Phase 5 — Translational Blur Refinement

The translational component (`deblur_use_translation`) is already implemented but uses an assumed constant scene depth. Phase 5 replaces this with a better depth estimate.

Option A — Known operating depth (easiest)

For a subsea rover at a known working distance from the surface being scanned, measure the typical standoff distance. Set `deblur_assumed_depth_m` to this value. This is already implemented and sufficient for most structured survey work.

Option B — Depth from stereo disparity

Since you have two calibrated cameras (after Phase 1 and stereo calibration), you can compute a dense depth map from stereo disparity and look up the scene depth at each tile centre:

1. Run stereo calibration with `camera_calibration`:

```
ros2 run camera_calibration cameracalibrator \
  --size 7x9 --square 0.025 \
  --approximate 0.1 \
  right:=/cam1/camera/image_raw left:=/cam0/camera/image_raw \
  right_camera:=/cam1/camera left_camera:=/cam0/camera
```

2. Rectify the image pair using `cv2.stereoRectify + cv2.initUndistortRectifyMap`.
3. Compute disparity with `cv2.StereoSGBM`.
4. Convert disparity to depth: $Z = (fx * baseline) / disparity$.
5. Sample depth at each tile centre and pass per-tile depth into `_deblur_tiled`.

Prerequisite: Camera baseline (distance between camera optical centres) measured physically or from Kalibr output.

Time estimate: 3–4 hours for stereo calibration and depth computation.

Phase 6 — Post-Processing with RTS Smoother

What it gives you: Better trajectory estimates for existing bags. The online EKF is causal — it only uses data up to time t . An RTS smoother runs forward then backward over the full bag, giving optimal estimates that use all available IMU and GPS data. This is most valuable when GPS signal was intermittent.

Implementation steps

1. Install GTSAM (C++) or use the Python `filterpy` RTS smoother:

```
pip3 install filterpy numpy scipy
```

2. Extract IMU and odometry from the bag into numpy arrays:

```
ros2 bag convert -i <bag_dir> -o extracted.db3
# or use mcap Python reader
pip3 install mcap mcap-ros2-support
```

3. Implement forward EKF + backward RTS pass:

```
from filterpy.kalman import KalmanFilter
# State: [x, y, z, vx, vy, vz, roll, pitch, yaw]
# Measurement: GPS position at 20 Hz
# Process model: IMU integration at 100 Hz
# Run forward pass → save all (x, P) pairs
# Run backward RTS pass → smoothed trajectory
```

4. For each capture timestamp in the bag, look up the smoothed velocity and use it for translational blur instead of the online EKF output.
5. Re-run deblurring offline on the saved raw images using the smoothed trajectory.

Time estimate: 4–6 hours for a working offline pipeline.

Phase 7 — Gyro Bias Estimation

What it gives you: Removes the constant drift offset from the gyro, reducing systematic blur direction error.

Procedure:

1. At the start of each session, hold the rover stationary for 10–30 seconds before moving.
2. From the bag, find this stationary period (where $|\text{linear_acceleration}| \approx 9.81 \text{ m/s}^2$ and $|\text{angular_velocity}|$ is minimal).
3. Average the gyro readings over this window — this is the bias vector $[b_x, b_y, b_z]$.
4. Add `deblur_gyro_bias_x/y/z` parameters and subtract the bias in `_integrate_gyro_interval` before integration:


```
dtheta_x += 0.5 * ((w_a[0]-bias_x) + (w_b[0]-bias_x)) * dt_s
```

For the BNO085, the internal sensor fusion already compensates for some gyro drift, so this may give minor improvement only.

Time estimate: 1–2 hours.

Implementation Priority

Phase	Quality Gain	Effort Do first?
1 — Intrinsic calibration	Medium	2–4 h Yes — unlocks Phase 3/4
2a — Time offset	Medium	2–3 h Yes
2b — Extrinsic (Kalibr)	High	3–4 h Yes
3 — Rotation matrix	Low at 8 ms	2–3 h After Phase 2
4 — Tiled homography PSF	High	4–6 h After Phase 1–3
5a — Fixed depth	Done	— Already done
5b — Stereo depth	Medium	3–4 h Later
6 — RTS smoother	Medium (post only)	4–6 h Later
7 — Gyro bias	Low (BNO085 compensates)	1–2 h Last

Recommended order: Phase 1 → Phase 2a → Phase 2b → Phase 3 → Phase 4 basic → Phase 4 with overlap blending.

Total to full 6DoF with tiled PSF: approximately **20–25 hours** of implementation and calibration time.

Key Parameters Added So Far

```
ros2 launch subsea_bringup rover_app.launch.py \  
  deblur_method:=wiener                \  
  deblur_wiener_snr:=40.0               \  
  deblur_imu_to_cam_yaw_deg:=0.0        \  
  deblur_use_translation:=true          \  
  deblur_assumed_depth_m:=1.5           \  
    survey depth                        \  
  deblur_max_odom_age_ms:=200.0         \  
  deblur_fov_deg:=72.0                  \  
  deblur_strength:=1.0                  \  
  magnetic_declination_radians:=0.076   \  
  deblur_imu_to_cam_yaw_deg:=0.0        # update after Phase 2b
```

Generated for trajectory-calculation-3d-scanner — May 2026